

A zone you own, answered over the wire

BIND9 serving a forward and a reverse zone, and a client that resolves through it across the lab LAN. Every block below was captured from a running lab — including the query itself, seen leaving the client and arriving at the server.

Command `qlab run dns-lab`

VMs `dns-lab-server 10.20.30.1 · dns-lab-client 10.20.30.50`

Zone `lab.qlab forward · 30.20.10.in-addr.arpa reverse`

Outcome `qlab test dns-lab` → 7 exercises, all passed

1. The two machines, and the address that is not a fiction

```
--- server ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          10.20.30.1/24 fe80::5054:ff:fe00:e01/64
--- client ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          10.20.30.50/24 fe80::5054:ff:fe00:e02/64
```

The lab LAN is `10.20.30.0/24`, and that is not an arbitrary choice: it is the subnet the zone file itself advertises. `ns1.lab.qlab` is declared as `10.20.30.1`, and the server really answers on `10.20.30.1`. So the record is true rather than decorative — you can resolve `ns1.lab.qlab` and then ping the result, and reach the machine that gave you the answer.

Each VM also carries QEMU's SLIRP card for `qlab shell`. Two interfaces on a DNS box is exactly the situation where "which address is my server listening on" stops being a rhetorical question.

2. What the server is authoritative for

```
zone "lab.qlab" {
    type master;
    file "/etc/bind/zones/db.lab.qlab";
};

zone "30.20.10.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.10.20.30";
};
```

Two zones: names to addresses, and addresses back to names. They are separate files with separate serial numbers, and nothing keeps them in agreement except you — which is why a reverse lookup so often disagrees with the forward one on real networks.

```
$TTL      604800
@         IN      SOA    ns1.lab.qlab. admin.lab.qlab. (
                        2024010101    ; Serial
                        3600           ; Refresh
                        1800           ; Retry
                        604800         ; Expire
                        86400 )        ; Negative Cache TTL
@         IN      NS     ns1.lab.qlab.
ns1       IN      A       10.20.30.1
web       IN      A       10.20.30.10
mail      IN      A       10.20.30.20
db        IN      A       10.20.30.30
app       IN      A       10.20.30.40
mail2     IN      A       10.20.30.21
web       IN      AAAA    2001:db8::10
www       IN      CNAME   web.lab.qlab.
ftp       IN      CNAME   web.lab.qlab.
@         IN      MX      10 mail.lab.qlab.
@         IN      MX      20 mail2.lab.qlab.
```

```
@      IN      TXT      "v=spf1 mx -all"
_dmarc IN      TXT      "v=DMARC1; p=reject"
_http._tcp IN     SRV      10 0 80 web.lab.qlab.
_mysql._tcp IN     SRV      10 0 3306 db.lab.qlab.
```

Everything a beginner meets in one place: `SOA` and its timers, `NS`, `A` and `AAAA`, `CNAME` aliases, `MX` with priorities, `TXT` carrying SPF and DMARC policy, and `SRV` advertising a service with its port.

```
active
named-checkconf: OK
zone lab.qlab/IN: loaded serial 2024010101
OK
```

`named-checkconf` and `named-checkzone` are the two commands worth remembering: BIND will refuse to load a zone with a syntax error and keep serving the previous one, so "I edited the file and nothing changed" is almost always a zone that never loaded.

3. A query crossing the network

The client asks; `tcpdump` on the server watches it arrive.

```
--- on the client ---
web.lab.qlab.      604800 IN  A   10.20.30.10
web.lab.qlab.

--- on the server: tcpdump -i any -n port 53 ---
09:23:08.216709 IP 10.20.30.50.53658 > 10.20.30.1.53: 12135+ [1au] A? web.lab.qlab. (53)
09:23:08.216932 IP 10.20.30.1.53 > 10.20.30.50.53658: 12135* 1/0/1 A 10.20.30.10 (85)
09:23:08.217088 IP 10.20.30.1.53 > 10.20.30.50.53658: 12135* 1/0/1 A 10.20.30.10 (85)
09:23:08.236666 IP 10.20.30.50.34485 > 10.20.30.1.53: 4534+ [1au] PTR? 10.30.20.10.in-addr.arpa. (65)
```

The capture is the whole lesson. A query leaves `10.20.30.50` from a random high port, arrives at `10.20.30.1` port 53, and the answer comes straight back. The `+` after the query id means recursion was requested; the `*` in the reply means the server is **authoritative** for that name — it did not go and ask anyone else, it simply knows.

The last line is the reverse lookup, and it shows the trick of `in-addr.arpa`: to ask about `10.20.30.10` you query the name `10.30.20.10.in-addr.arpa`, with the octets reversed, because DNS delegates from the right and addresses are written most-significant-first.

4. Record types, from the client

A	web.lab.qlab	10.20.30.10
AAAA	web.lab.qlab	2001:db8::10
CNAME	www.lab.qlab	web.lab.qlab.
MX	lab.qlab	20 mail2.lab.qlab. 10 mail.lab.qlab.
TXT	lab.qlab	"v=spf1 mx -all"
SRV	_http._tcp.lab.qlab	10 0 80 web.lab.qlab.
NS	lab.qlab	ns1.lab.qlab.

`MX` returns two hosts with priorities — lower wins, and `mail2` at 20 is the fallback. `SRV` carries a port, which is how a client finds a service without anyone hardcoding `:80`. `CNAME` returns another *name*, not an address, so the resolver has to go round again.

5. The search domain, which behaves differently in every tool

```
$ dig www +short
(empty: dig ignores the search list)
$ dig +search www +short
web.lab.qlab.
10.20.30.10
$ host www
www.lab.qlab is an alias for web.lab.qlab.
web.lab.qlab has address 10.20.30.10
web.lab.qlab has IPv6 address 2001:db8::10
```

This trips people up constantly. `dig` ignores `search` in `/etc/resolv.conf` unless you pass `+search`; `host` and `nslookup` apply it. So `dig www` returning nothing while `host www` works is not a broken server — it is two tools with different defaults.

How the client resolves

```
nameserver 127.0.0.1
search lab.qlab

# Forward every query to the lab's authoritative server, straight over
# the lab LAN. (This used to point at 10.0.2.2#5354 – the host, on a
# hardcoded port – but QLab allocates host ports dynamically, so that
# number was almost never the right one and resolution simply hung.)
no-resolv
server=10.20.30.1
# Add lab.qlab to search domain
domain=lab.qlab
```

The client points at itself, and a local `dnsmasq` forwards everything to `10.20.30.1`. That is a small cache in front of the authoritative server, which is how most real networks are arranged.

6. Verification

```
Exercise 1: All 5 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 2 checks passed
Exercise 4: All 2 checks passed
Exercise 5: All 3 checks passed
Exercise 6: All 3 checks passed
Exercise 7: All 13 checks passed
Exercises run:      7
```

```
Exercises passed: 7
Exercises failed: 0
Exercises skipped: 0
All exercises passed!
```

Exercise 7 is the one that checks the network: exercises 1 to 6 query the server *from the* server with `@localhost`, and would pass even if the two machines could not reach each other at all.

7. What to take away

- Forward and reverse are two independent zones. Nothing synchronises them.
- An authoritative answer (`*` in a capture, `aa` in a full `dig`) means the server knows, rather than having asked upstream. It is the difference between serving a zone and being a resolver.
- `in-addr.arpa` reverses the octets because DNS delegates from the right.
- `dig` does not use the search list unless told to. `host` does.
- If an edit seems to have no effect, run `named-checkzone` : BIND keeps serving the last zone that loaded cleanly.

`guide.md` in the plugin goes further: adding records and bumping the serial, reverse delegation, and reading a resolution chain with `+trace` .